

Análisis de Patrones de Diseño y Arquetipos

Proyecto Sanos y Salvos — Evaluación Parcial N°2

DSY1106 Desarrollo Fullstack III

Integrante	Rol
Benjamin Arellano Gallardo	Desarrollador Fullstack
Benjamin Valdebenito	Desarrollador Fullstack
Maximiliano Vera	Desarrollador Fullstack
Matias Guzman	Desarrollador Fullstack

1. Introducción

El presente documento describe los patrones de diseño seleccionados e implementados en el proyecto Sanos y Salvos, una plataforma de gestión veterinaria desarrollada con arquitectura de microservicios Spring Boot, frontend React y autenticación mediante Keycloak. La selección de cada patrón responde a necesidades concretas del sistema, buscando garantizar la mantenibilidad, escalabilidad y coherencia del código.

Adicionalmente, se describen los arquetipos Maven creados como base reutilizable para la generación de nuevos componentes backend, siguiendo los estándares definidos para el proyecto.

2. Patrones de Diseño

2.1 Patrón Builder — Microservicio Mascotas

Descripción: El patrón Builder separa la construcción de un objeto complejo de su representación, permitiendo crear diferentes representaciones usando el mismo proceso de construcción.

Problema que resuelve:

- El objeto Mascota contiene múltiples atributos opcionales (nombre, especie, raza, edad, peso, estado de salud). Sin un patrón estructurado, los constructores se vuelven difíciles de mantener y propensos a errores por el orden de los parámetros.

Implementación:

- MascotaBuilder permite construir objetos Mascota de forma fluida mediante method chaining, validando los campos requeridos antes de construir el objeto final.
- La clase Mascota expone un builder estático interno que garantiza que solo se puedan crear instancias válidas.

Justificación:

- Mejora la legibilidad del código, facilita la creación de objetos en pruebas unitarias y permite agregar nuevos atributos sin romper el código existente.

2.2 Patrón Facade — Microservicio Reportes y BFF

Descripción: El patrón Facade provee una interfaz simplificada a un conjunto de interfaces en un subsistema, facilitando su uso y reduciendo dependencias.

Problema que resuelve:

- El BFF necesita coordinar llamadas a múltiples microservicios para componer respuestas complejas. Sin Facade, esta lógica se dispersa en los controladores, dificultando el mantenimiento.

Implementación:

- SanosYSalvosFacade en el BFF centraliza la orquestación de llamadas, exponiendo métodos de alto nivel como `getMascotaConPropietario()` y `getMascotaConReportes()`.
- `MascotaServiceFacade` en `msvc-reportes` simplifica el acceso a datos de mascotas desde el contexto de los reportes.

Justificación:

- Reduce el acoplamiento entre el frontend y los microservicios. Centraliza la lógica de composición. Facilita cambios internos sin afectar a los consumidores.

2.3 Patrón Adapter — Microservicio Usuarios

Descripción: El patrón Adapter permite que interfaces incompatibles trabajen juntas, actuando como puente entre dos interfaces.

Problema que resuelve:

- El microservicio de usuarios necesita consumir datos del microservicio de mascotas, pero los modelos de datos no son directamente compatibles.

Implementación:

- `MascotaClientAdapter` adapta las respuestas del cliente HTTP de mascotas al modelo interno del microservicio de usuarios mediante un `MascotaDTO` intermedio.

Justificación:

- Desacopla los modelos de datos entre microservicios, permitiendo que cada uno evolucione independientemente.

3. Arquetipos Maven

Se desarrollaron dos arquetipos Maven que encapsulan la estructura estándar del proyecto, permitiendo generar nuevos componentes de forma consistente y rápida.

3.1 arquetipo-microservicio

Genera la estructura base de un microservicio Spring Boot con integración a Eureka, configuración JPA con H2, y las capas controller, service y repository listas para implementar. Incluye configuración de pruebas unitarias con JUnit 5.

3.2 arquetipo-bff

Genera la estructura base de un Backend For Frontend con integración a Eureka y WebFlux para llamadas reactivas a los microservicios. Incluye la estructura de carpetas client, controller y facade siguiendo el patrón Facade del proyecto.

4. Autenticación con Keycloak

La plataforma integra Keycloak como servidor de autenticación y autorización OAuth2/OpenID Connect. Se levanta mediante Docker con un realm preconfigurado que se importa automáticamente, eliminando la necesidad de configuración manual.

Propiedad	Valor
Realm	sanosysalvos
Client ID	sanos-y-salvos-client
Puerto Keycloak	9090
Redirect URI	http://localhost:5173/*
Usuario de prueba	usuario-test / admin123

El archivo `keycloak/realm-export.json` incluye el realm, el client y el usuario de prueba preconfigurados. Al ejecutar `docker compose up -d`, Keycloak importa automáticamente esta configuración, dejando el sistema listo para autenticarse sin pasos adicionales.

5. Conclusión

Los tres patrones seleccionados (Builder, Facade y Adapter) responden directamente a problemas concretos de la arquitectura de microservicios implementada. Su combinación permite un sistema desacoplado, mantenible y extensible. La integración de Keycloak via Docker garantiza una autenticación robusta y reproducible en cualquier entorno de desarrollo. Los arquetipos Maven aseguran que cualquier extensión futura del sistema mantenga la coherencia estructural y los estándares de calidad definidos.